Neural networks, from the perceptron up

Fashion-MNIST

LECTURE 9 GÉRON CH. 9

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Part II begins

Part	Lectures	Chapters	Hardware
I · Tabular data and classical models	1–8	1–8	CPU
II · Neural networks	**9–11**	**9–11**	**CPU**
III · Computer vision	12–14	12	CPU
IV · Sequences and language	15–18	13–15	CPU
V · Search and recommendation	19–22	—	CPU
VI · Multimodal models	23–24	15–16	CPU

Every notebook in this course runs on Colab’s free CPU tier. The corpora are subsampled and the networks are small enough to train in minutes — deliberately, so that every result on a slide is one you can reproduce rather than one you must take on trust.

Today

1. The task, the corpus, and why a pixel is a feature (10 min)
2. **The mathematics** — what a layer computes (20 min)
3. **The method** — the perceptron, the multilayer perceptron, activations, and the architecture tables (40 min)
4. What `MLPClassifier` cannot do, and why the next lecture exists (15 min)

PART ONE

The brief

15 minutes · the problem, the data, the stakeholder

The stakeholder

A document-processing bureau scans incoming items and routes each one to one of **ten** queues.

- Items arrive as small greyscale scans, already cropped and centred
- A human currently sorts them; the operator wants to know how often a machine would be right
- Misrouting is recoverable but expensive: the item goes to the wrong desk and comes back a day later

Ten classes, one label each, no ranking, no probability requirement stated. Write that down: it decides the output layer.

This is not the classifier you built in Lecture 3

LECTURE 3–4

- Two classes
- Wildly imbalanced
- Accuracy was a trap
- The interesting question was *where to put the threshold*

TODAY

- Ten classes
- Exactly balanced
- Accuracy is defensible — we will check, not assume
- The interesting question is *what function to fit*

A different problem needs a different metric. The habit worth keeping from Lecture 4 is not “accuracy is bad”; it is “look at the class counts before you choose”.

The corpus we stand in with

The bureau's archive is confidential. **Fashion MNIST** is the dataset Chapter 9 uses, and it has the same shape of problem: small greyscale images, ten classes, one label each.

Property	Value
Training images	60,000
Test images (shipped as a test set)	10,000
Image size	28 × 28
Features per image, flattened	784
Pixel values	0–255, integer
Classes	10

The test split is the one the dataset ships with, so your number is comparable with every published result on it.

Ten classes, four examples each

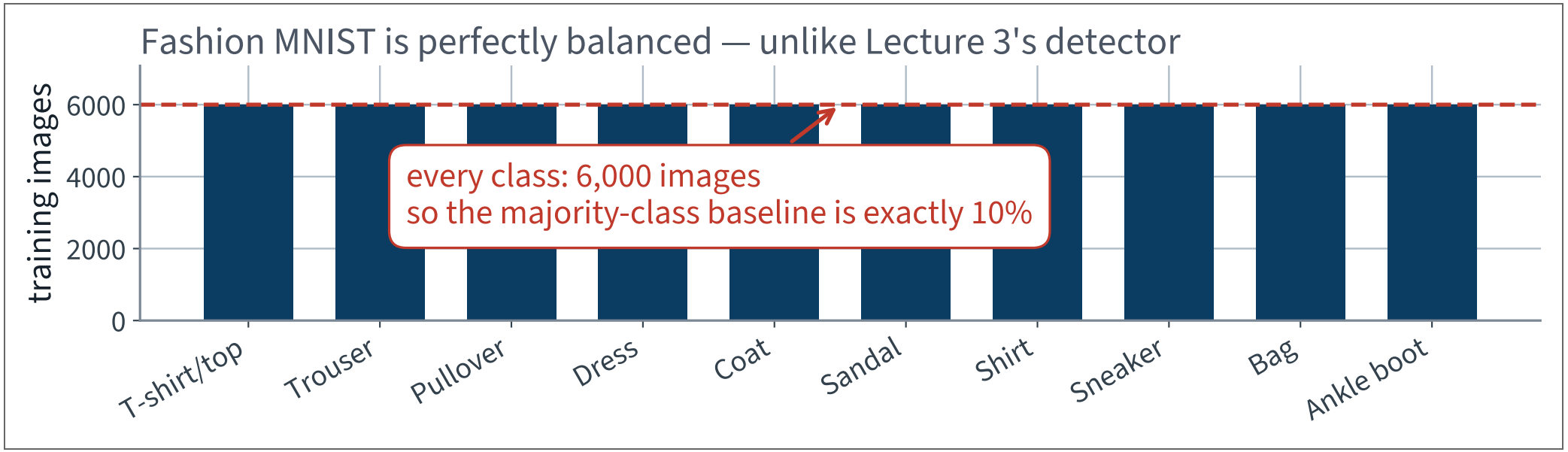
Pullover, coat and shirt are one silhouette at this resolution — and that is where the errors will be.

The ten classes

1. T-shirt/top
2. Trouser
3. Pullover
4. Dress
5. Coat
6. Sandal
7. Shirt
8. Sneaker
9. Bag
0. Ankle boot

Three of these are upper-body garments seen from the front. Ask the room which three, before the model tells you.

Check the balance before choosing the metric



Ten bars, all exactly 6,000. Balanced by construction — which is a property of this dataset, not of image classification.

What the data does not contain

- **No colour.** A single greyscale channel, so a red shirt and a blue shirt are the same image
- **No scale.** Every garment fills the frame, so the model cannot use size to tell a sandal from a boot
- **No context.** One item per image, centred, on a black background

> **WHY THAT IS WORTH SAYING OUT LOUD**
>
> Every one of those is a simplification the bureau's real scans will not honour. A number measured here is an upper bound on what the same method gives on the archive, and the report should say so.

A pixel is a feature

We flatten each 28 × 28 image into a vector of 784 numbers and hand it to the model as an ordinary table of features.

> **WHAT THAT THROWS AWAY**
>
> The model is told nothing about which pixels are neighbours. Shuffle all 784 columns with one fixed permutation and it learns exactly as well. A human could not read a single shuffled image.

That is a real loss, and repairing it is Chapter 12 — four lectures away. Today we pay it, and it is the honest starting point.

Scale the pixels

Divide by 255, so every feature lies in $[0, 1]$.

- Networks are trained by gradient descent, and the step size is one number shared by every weight
- Inputs two orders of magnitude apart make one step too large for some weights and too small for others

> **AND NO, THIS IS NOT LECTURE 2'S LEAK**
>
> Dividing by a constant that is a property of the file format fits nothing. `StandardScaler` estimates a mean and a variance *from data* and must therefore be fitted after the split. `x / 255` estimates nothing.

The split

Set	Images	Used for
Training	55,000	fitting the weights
Validation	5,000	choosing hyperparameters, by hand, today
Test	10,000	once, at the very end

The validation set is carved out of the training half with a fixed seed. It exists because we are about to try ten configurations, and Lecture 2 established what happens to a number chosen on the set it is reported on.

THE MATHEMATICS

What a layer computes

20 minutes · examinable

Where the idea came from

McCulloch and Pitts, 1943: a very simplified model of a biological neuron, which fires when enough of its inputs fire.

> *The analogy is historical and it is where the vocabulary comes from — neuron, activation, firing. It is not an argument for anything. Modern networks resemble brains about as closely as aeroplanes resemble birds.*

Chapter 9 opens with this and then drops it. So do we.

The threshold logic unit

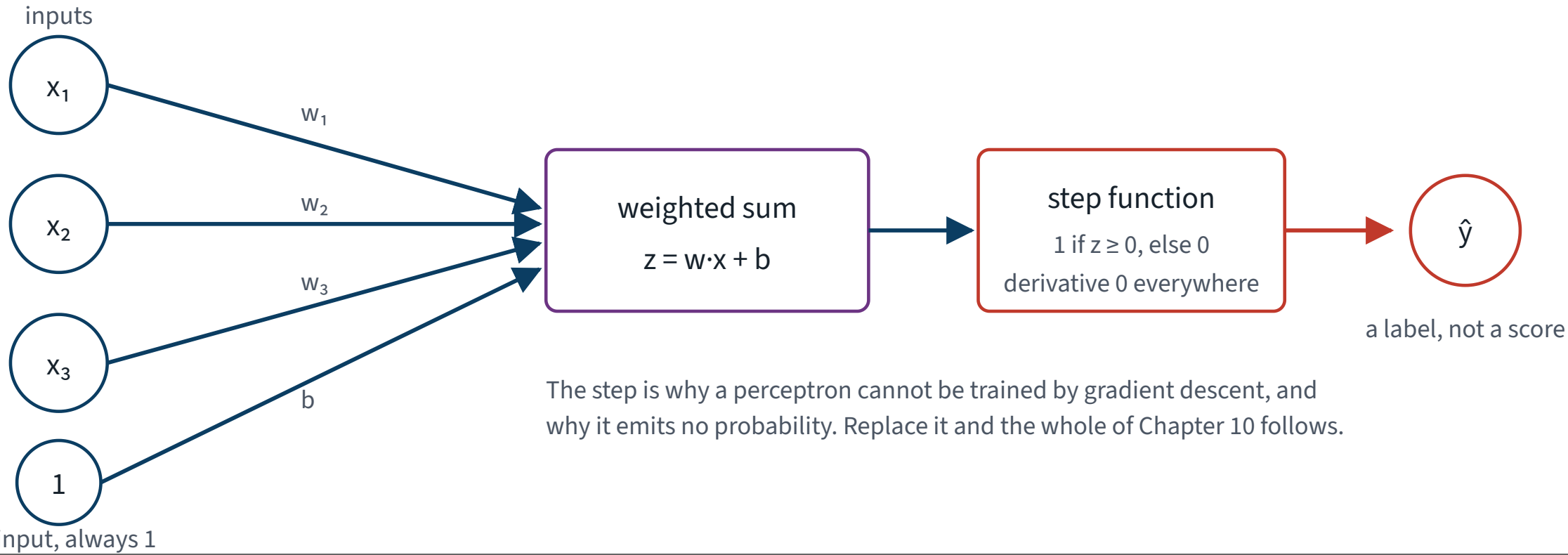
$$z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b, \qquad \hat{y} = \operatorname{step}(z)$$

- $\mathbf{x}$ — the input features, one number each
- $\mathbf{w}$ — one weight per input, learned
- b — the bias, learned; equivalently a weight on a constant input of 1
- step — 1 if $z \geq 0$, else 0

A weighted sum and a threshold. That is the entire unit, and the weighted sum is Lecture 2's linear model wearing a different name.

One unit, drawn

One threshold logic unit — the perceptron's only moving part



✗ The step function is the problem. Its derivative is zero everywhere it exists, so there is no gradient to descend.

How a perceptron was trained

PERCEPTRON LEARNING RULE

$$w_{i,j}^{\text{next}} = w_{i,j} + \eta\left(y_j - \hat{y}_j\right) x_i$$

If the unit was right, nothing changes. If it fired when it should not have, the weights on the active inputs go down. Rosenblatt, 1957 — and it converges only if the classes are linearly separable.

No loss function appears anywhere in that rule. That is the tell: this is not gradient descent, and it does not generalise.

What a single unit cannot do

A TLU draws **one straight line**. Some problems need two.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

No line separates the two 1s from the two 0s. Minsky and Papert said so in 1969 and the field stopped for fifteen years.

✓ The repair is embarrassingly simple: stack them. Two layers of TLUs solve XOR exactly.

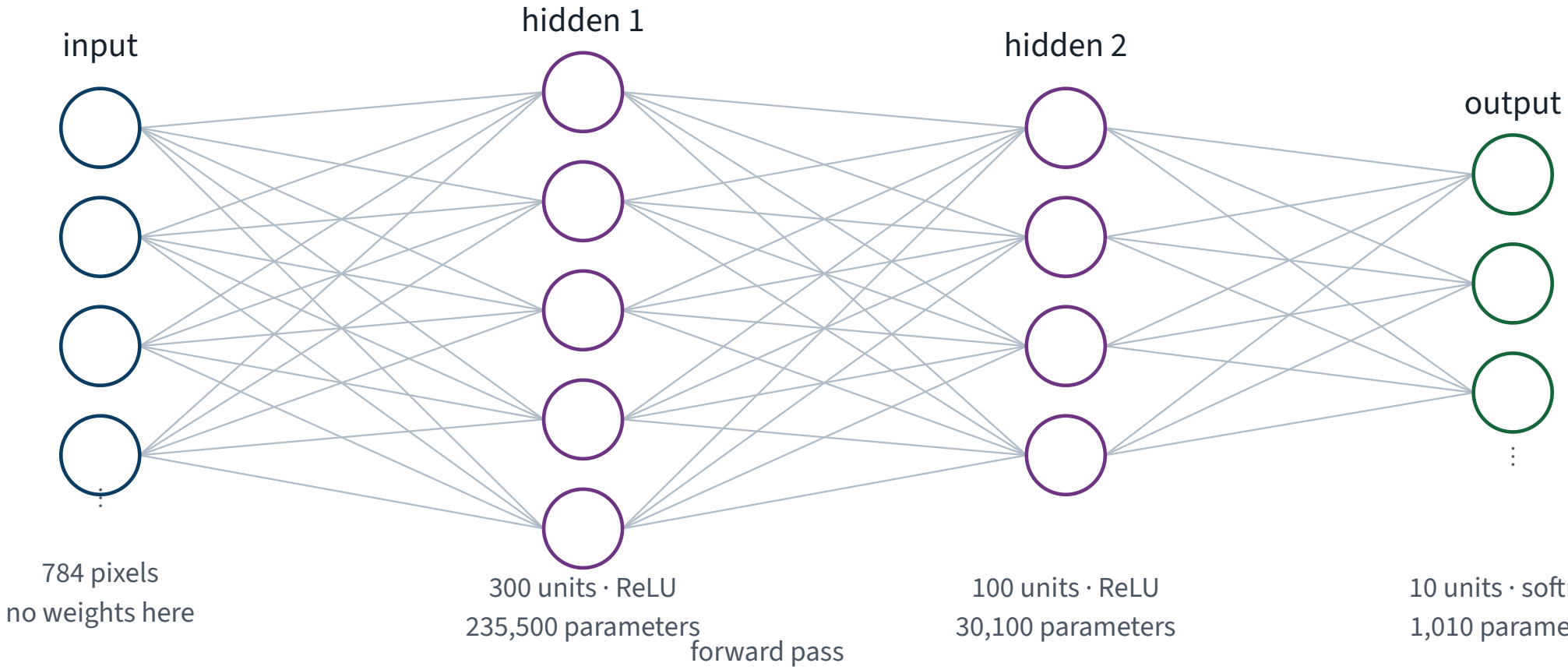
The multilayer perceptron

- An **input layer**, which holds the features and has no weights
- One or more **hidden layers**, each fully connected to the previous one
- An **output layer**, one unit per class

Fully connected, or *dense*: every unit of one layer receives every unit of the previous one. That is where the parameters are.
Two or more hidden layers and the book calls it a **deep** neural network. The word means nothing more than that.

The network this application builds

The multilayer perceptron this application builds



Almost all the parameters are in the first layer, because it is the one attached to 784 inputs.

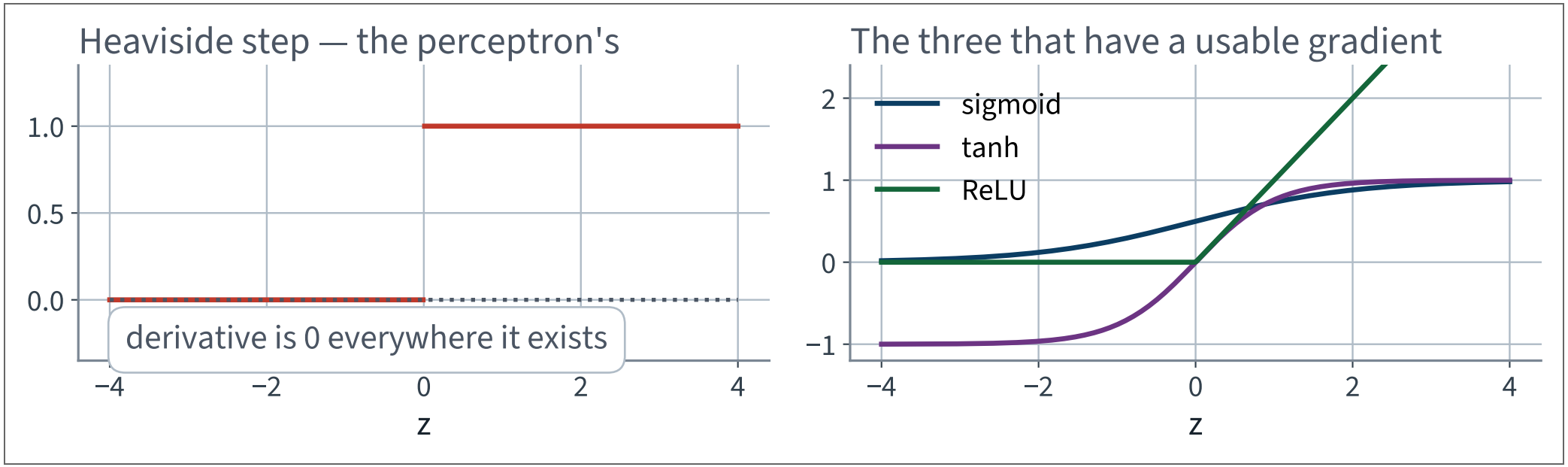
Why there must be a nonlinearity

Suppose the activation were the identity. Then

$$\mathbf{W}_2\left(\mathbf{W}_1\mathbf{x}+\mathbf{b}_1\right)+\mathbf{b}_2 \;=\; \left(\mathbf{W}_2\mathbf{W}_1\right)\mathbf{x}+\left(\mathbf{W}_2\mathbf{b}_1+\mathbf{b}_2\right)$$

✗ A single affine map. Two hundred stacked layers would still be a single affine map.
Depth buys you nothing without a nonlinearity between the layers. This is one line of algebra and it is the reason activation functions exist at all.

The step had to go



Gradient descent needs a slope to descend. The step has none; the other three do.

Why ReLU won

Activation	Range	Cost	Problem
Sigmoid	$(0, 1)$	an exponential	saturates at both ends
tanh	$(-1, 1)$	an exponential	saturates at both ends
✓ ReLU	**✓** $[0, \infty)$	**✓ a comparison**	**✓ flat below zero**

`max(0, z)` is fast, does not saturate above, and in practice trains better. It is the default in Chapter 10 and it is our default today.

“Flat below zero” is a real defect and it has a name — dying ReLUs. It is diagnosed in Lecture 14, not here.

One layer, derived

1. A single unit computes $z = \mathbf{w}^\mathsf{T}\mathbf{x} + b$, a weighted sum of its inputs and one offset.

   Two numbers per unit and no more: a direction, and how far along it to sit

2. Stack n units side by side and the weights become a matrix: $\mathbf{z} = \mathbf{W}^\mathsf{T}\mathbf{x} + \mathbf{b}$, with $\mathbf{W}$ one column per unit.

   The layer is one matrix product, which is why a tuned BLAS matters at all

3. Feed a whole mini-batch at once by making $\mathbf{X}$ one row per instance: $\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b}$.

   Shapes: $(m \times d)(d \times n) \to (m \times n)$, and $\mathbf{b}$ broadcasts along the rows

4. Apply a non-linearity elementwise: $\mathbf{H} = \phi(\mathbf{X}\mathbf{W} + \mathbf{b})$.

   Elementwise, so it changes no shape — the arithmetic above is untouched

Now remove ϕ, and watch the stack collapse

1. [5] Compose two layers with no non-linearity between them: $(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 = \mathbf{X}(\mathbf{W}_1\mathbf{W}_2) + (\mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2)$.

   Matrix product is associative, so the pair collapses

2. [6] That is $\mathbf{X}\mathbf{W}' + \mathbf{b}'$ — **one affine map**. A stack of linear layers, however deep, is a linear model.

   So ϕ is not a refinement. Without it there is no second layer at all

Derive the collapse, and state what it implies about depth.

EXAMINABLE

What that derivation buys you

- A layer is **one matrix product and one elementwise function**. Everything else is bookkeeping
- The parameter count is $d \times n + n$ per layer, so you can price a network before fitting it
- Depth is only worth having because ϕ is there; remove it and any stack collapses to a single affine map
- The shapes compose, which is what makes a mini-batch free

> **WHERE IT RETURNS**
>
> Lecture 10 differentiates exactly this expression, once per layer, backwards. Lecture 11 asks what happens to the variance of $\mathbf{H}$ as the stack gets deeper — and answers it with the same two objects, $\mathbf{W}$ and ϕ.

THE METHOD

From one unit to a network

40 minutes · examinable

One layer, in matrices

A WHOLE MINI-BATCH AT ONCE

$$\mathbf{H} = \phi(\mathbf{X}\mathbf{W} + \mathbf{b})$$

- $\mathbf{X}$ — one row per instance in the batch, one column per input
- $\mathbf{W}$ — one row per input, one column per unit
- $\mathbf{b}$ — one entry per unit, broadcast down the rows
- ϕ — the activation, applied element by element

The whole forward pass is a handful of matrix products. That is why a GPU helps at all: a GPU is a machine for doing exactly this.

Count the parameters before you fit

```
784 x 300 + 300  =  235,500     layer 1
300 x 100 + 100  =   30,100     layer 2
100 x  10 +  10  =    1,010     output
                   --------
                    266,610     total
```

266,610 parameters, fitted from 55,000 images.

> **FOUR AND A HALF PARAMETERS PER TRAINING IMAGE**
>
> Lecture 2's failure condition for least squares was $m < n$ — more parameters than instances. We are past it, comfortably, and the model will still work. Understanding why is Lectures 13 and 14.

The output layer

Ten classes, mutually exclusive. Ten output units, and a **softmax** across them:

$$\hat{p}_k = \frac{\exp(z_k)}{\sum_{j=1}^{10} \exp(z_j)}$$

- Every output is positive and they sum to 1
- The largest logit z_k gives the largest probability, so `argmax` is unchanged by the softmax

The full properties of softmax — shift invariance, the gradient, why the loss consumes logits rather than probabilities — are derived in Lecture 17.

The loss

CROSS-ENTROPY, ONE INSTANCE

$$L = -\sum_{k=1}^{10} y_k \log \hat{p}_k = -\log \hat{p}_{\text{true}}$$

The target is one-hot, so nine of the ten terms are zero. The loss is minus the log probability the model assigned to the right answer, and it goes to infinity as that probability goes to zero.

Chapter 4 called this log loss when there were two classes. Same object.

How the weights get updated

Backpropagation: one forward pass to compute the loss, one backward pass to compute every partial derivative, one gradient-descent step.

- Rumelhart, Hinton and Williams, 1986 — and the reason the field restarted
- The backward pass costs about as much as the forward one, no matter how many parameters there are

> **WHY “NO MATTER HOW MANY” IS REMARKABLE**
>
> 266,610 parameters, one number of interest, and a single sweep gets all of the derivatives. That claim is derived at the top of the next lecture, and it is the whole reason any of this is feasible.

The optimiser

`solver="adam"`. Plain gradient descent takes a step proportional to the gradient; Adam keeps a running average of the gradient and of its square, and scales each parameter's step by the second.

- Far less sensitive to the initial learning rate than plain SGD
- Which is why `learning_rate_init` still cost us points on the previous sweep — less sensitive is not insensitive

Momentum, RMSProp, Adam and the rest are Chapter 11 and Lecture 14. Today it is a default we accepted knowingly, which is the only acceptable way to accept a default.

Architecture — regression networks

Hyperparameter	Typical value
Input units	one per input feature
Hidden layers	1 to 5, problem dependent
Units per hidden layer	10 to 100, problem dependent
Output units	one per prediction dimension
Hidden activation	ReLU
✓ Output activation	**✓ none; ReLU if the output must be positive; sigmoid or tanh for a bounded range**
✓ Loss	**✓ MSE, or Huber if there are outliers**

Chapter 10. The two rows that matter are the last two, and they are the two people get wrong.

Architecture — classification networks

Task	Output units	Output activation	Loss
Binary	1	sigmoid	binary cross-entropy
Multilabel binary	one per label	sigmoid	binary cross-entropy
✓ Multiclass	**✓ one per class**	**✓ softmax**	**✓ cross-entropy**

Our task is the third row: ten mutually exclusive classes, so ten units, softmax, cross-entropy.

The distinction between rows two and three is not cosmetic. Multilabel asks ten independent yes/no questions; multiclass asks one ten-way question. Softmax couples the outputs; ten sigmoids do not.

How to read those two tables

They answer one question each, and the answers are forced rather than chosen:

Question	What decides it
How many output units?	the shape of the answer — one per target, one per class
Which output activation?	the range the answer must live in — none, sigmoid, softmax
Which loss?	what a wrong answer costs, and it must pair with the activation

> **THE PAIRING IS NOT OPTIONAL**
>
> Softmax with mean squared error trains, slowly and badly: the gradient through a saturated softmax nearly vanishes, and MSE does nothing to undo it. Cross-entropy is the loss that cancels the softmax derivative, which Lecture 17 derives. Until then, take the pairing from the table.

How wide, and how deep?

- **Depth** buys parameter efficiency: two narrow layers express things one wide layer needs exponentially many units for
- **Width** buys capacity within a layer, at $d \times n + n$ parameters a time
- In practice: pick a depth that is enough, then grow the width until validation stops improving — and stop there

> **THE HONEST ANSWER**
>
> There is no formula. What there is, is a budget you can compute before fitting anything — the parameter count from the derivation — and a validation curve that tells you when you have spent it badly.

The numbers depend on the corpus. The method for finding them does not.

NOT EXAMINABLE — ENGINEERING

PART TWO

Choose a metric, and its baseline

15 minutes

Which metric?

The operator asked: *how often is it right?*
That is accuracy. Lecture 4 spent ninety minutes on why accuracy is a trap — so before using it, check the condition under which it failed.

Lecture 4	Today
One class held 10%% of the data or worse	Every class holds exactly 10%%
“Always negative” scored above 90%	“Always *k*” scores 10.0%%
Accuracy hid a useless model	Accuracy cannot hide one

The rule you keep is not “never use accuracy”. It is **count the classes first**.

The trivial baseline, computed

```
majority = np.bincount(y_fit).argmax()
baseline = np.full(len(y_test), majority)
print(accuracy_score(y_test, baseline))       # 0.1
```

Ten classes, 1,000 test images each, one guess for every image.

Not an estimate and not a convention — a measurement, and it happens to be exact.

The whole model, in Scikit-Learn

```
from sklearn.neural_network import MLPClassifier

clf = MLPClassifier(hidden_layer_sizes=(300, 100),
                    activation="relu",
                    solver="adam",
                    learning_rate_init=1e-3,
                    batch_size=128,
                    max_iter=20,
                    random_state=42)
clf.fit(X_fit, y_fit)
```

Nine lines, and every architectural decision from the last ten slides is one of the arguments. The softmax and the cross-entropy are not arguments — `MLPClassifier` chooses them for you, from the number of classes it sees in `y_fit`.

The defaults you did not ask for

Argument	Default	Consequence
`learning_rate_init`	0.001	tuned for inputs of order one
`max_iter`	200	epochs, not iterations; the name is wrong
`batch_size`	min(200, n)	silently different on a small set
`early_stopping`	False	runs the full budget regardless
`n_iter_no_change`	10	stops early anyway, via `tol`

✗ Reviewer question 5, every time: *what is the default I did not ask for?* Two of these five will bite us within the hour.

Three words that get confused

Word	Means	Here
Batch	the instances used for one weight update	128
Step, or iteration	one weight update	430 per epoch
Epoch	one complete pass over the training set	20 of them

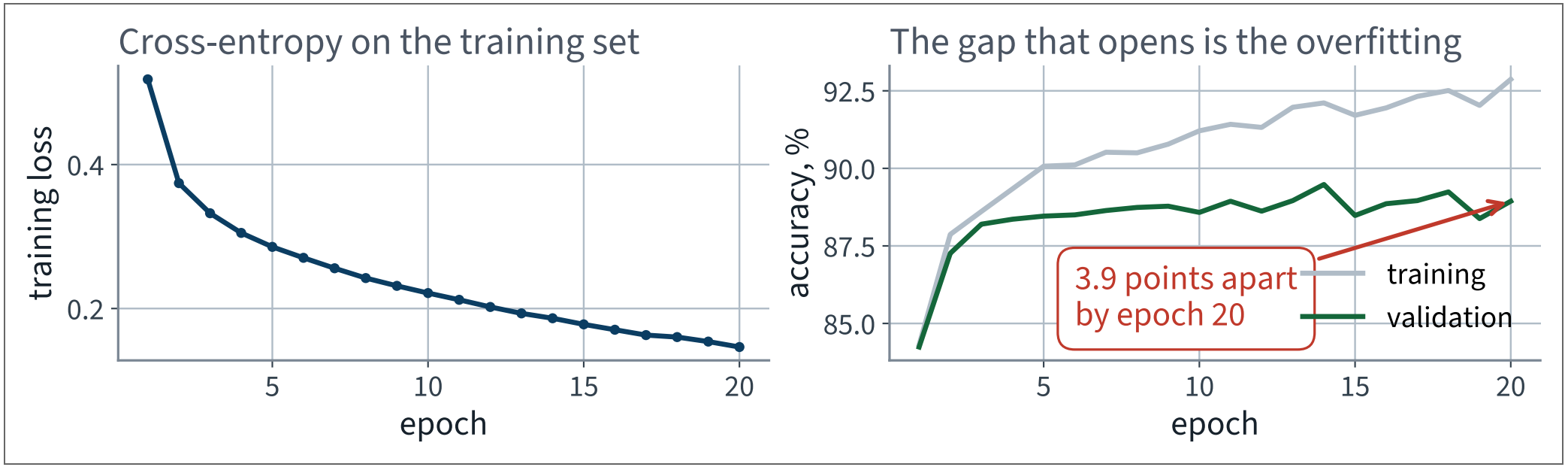
✗ Scikit-Learn calls the number of epochs `max_iter`. It is not iterations. Read the docstring, not the parameter name — and this is not the only library that does it.

Run it

Quantity	Measured
Training images	55,000
Epochs	20
Wall clock, CPU	125 s
Validation accuracy	88.94%%
✓ Test accuracy	**88.02%% ✓**
Baseline	10.0%%

It works. Hold on to the wall clock — it is the number the next lecture repairs.

What twenty epochs look like



✗ The two accuracy curves separate and stay separated. That gap is overfitting, and it is Lecture 2's table drawn as lines.

Reading the curve

- The loss falls smoothly — the optimiser is working
- Training accuracy keeps climbing — the model has capacity to spare
- Validation accuracy flattens — the extra capacity is being spent on memorising

> **THE QUESTION A CURVE ANSWERS THAT A NUMBER CANNOT**
>
> *Would more epochs help?* Here, no: the validation curve stopped moving before the budget ran out. Without the curve you would have guessed.

What we are *not* doing about that gap

Repair	Available today	Taught in
Stop when validation stops improving	`early_stopping=True`	Lecture 6, applied here
Penalise large weights	`alpha`	Lecture 6
Dropout	**✗ not available**	Lecture 10, then 11
Batch normalisation	**✗ not available**	Lecture 14
More data, or augmented data	**✗ not available**	Lecture 16

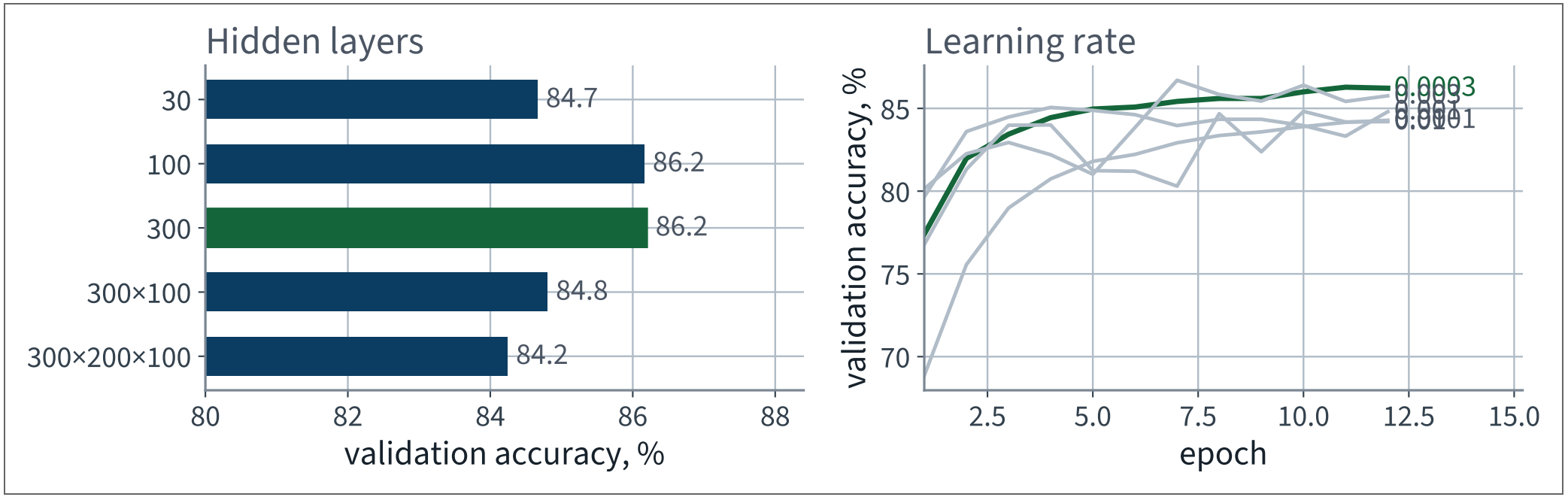
Three of the five repairs are not reachable from this library at all. Note which ones, and why: they all need a hook inside the training loop.

What there is to turn

Knob	What it changes
Number of hidden layers	how many compositions the function can have
Units per layer	how much can be represented at each stage
✓ Learning rate	**✓ the size of every gradient step**
Batch size	how noisy each step is, and how fast an epoch runs
Epochs	how long before you stop

Ten fits, by hand, on a 10,000-image subset so they fit in the lecture. Five architectures, then five learning rates.

Ten fits, by hand



✗ Both panels span about two accuracy points. Neither knob dominates — and the widest architecture is not the winner.

Architecture

Hidden layers	Parameters	Validation accuracy	Seconds
30	23,860	84.66%%	169
100	79,510	86.16%%	225
✓ 300	238,510	**86.20%% ✓**	347
300, 100	266,610	84.80%%	555
300, 200, 100	316,810	**✗ 84.24%%**	841

✗ On this subset, depth loses. One hidden layer of 300 beats both deeper networks, and the deepest is the worst and the slowest.

Learning rate

Learning rate	Validation accuracy
0.0001	84.28%%
✓ 0.0003	**86.22%% ✓**
0.001 — the default	84.80%%
0.003	85.76%%
0.01	**✗ 84.20%%**

Same architecture, same data, same seed. The only thing that changed is the size of the step — and **the library's default is not the best value on this grid.**

The lesson from ten fits

1. **Depth did not pay.** On 10,000 images a single hidden layer of 300 beat both deeper networks — and took less than half the time of the deepest.
2. **Neither knob dominates.** The five architectures span about two accuracy points; so do the five learning rates. Anyone who tells you one of them "matters more" has not measured it on your problem.
3. **Ten fits is not a search.** Ten points from a space with five axes, varied one axis at a time with everything else held fixed.

Point 3 is the one to remember. You have not optimised anything; you have looked at ten places.

And the sweep does not transfer

Run	Images	Epochs	Accuracy
Best in the sweep — 300, at lr 0.0003	10,000	12	86.20%% validation
✓ The headline model — 300, 100, at lr 0.001	55,000	20	**88.02%% test ✓**

✗ The architecture the sweep ranked *fourth of five* is the one that wins with all the data.

> **THE RULE**
>
> A deeper network needs more data to pay for its extra parameters. Rank architectures on a fifth of your data and you will systematically prefer the small ones — and never find out.

An accuracy is one number over ten classes

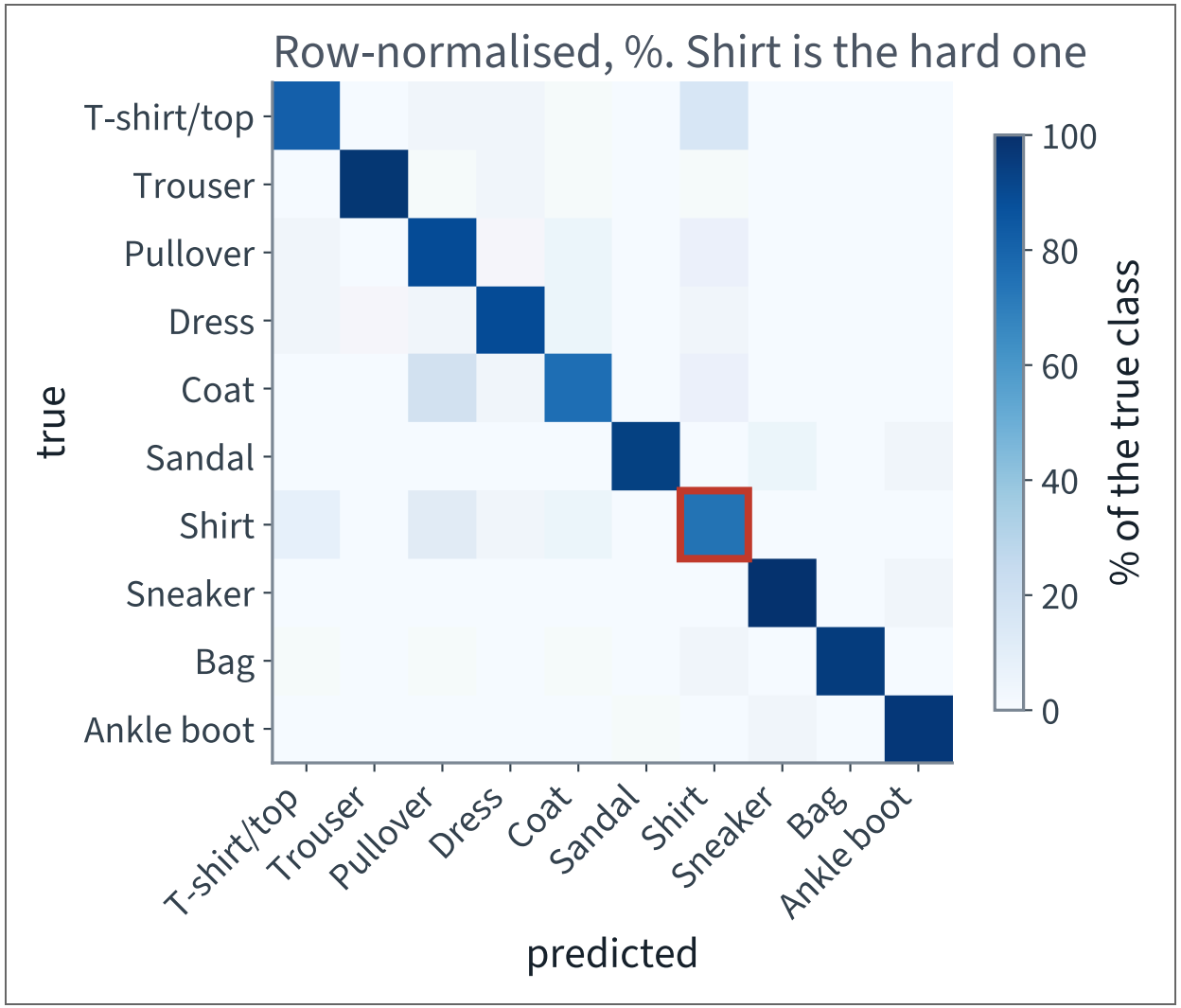
Split it. Lecture 4 built the confusion matrix for two classes; the object generalises without change.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, clf.predict(X_test))
recall = cm.diagonal() / cm.sum(axis=1)      # one per class
```

Ten rows, ten columns. Row i, column j counts the images of class i the model called class j.

Where the errors are



✗ Two classes are far worse than the other eight, and their errors go to a handful of specific neighbours.

The hard class

Recall on **Shirt**: **✗ 74.1%%**, against 98.1%% on Sneaker.

Shirts are called…	Share of all shirts
Pullover	10.9%%
T-shirt/top	7.8%%
Coat	4.6%%

Exactly the three classes the room named twenty minutes ago from the image grid — and Coat is nearly as bad, at 74.8%%. The model is not confused about anything a person would find easy.

What to tell the operator

> **A DEFENSIBLE REPORT**
>
> “Overall it is right 88.0%% of the time. But it is right 74.1%% of the time on Shirt, and almost all of those errors go to three visually similar queues. If those three desks are adjacent, the practical error rate is much lower than the headline. If they are in different buildings, it is much worse.”

The headline accuracy does not answer the operator's question. The per-class breakdown does, and it took one extra function call.

It works

88.0%

Against a 10.0% baseline, on 10,000 test images the model has never seen.

Test set touched once. Do not tune against it now.

THE LAST FIFTEEN MINUTES

What `MLPClassifier` cannot do

15 minutes

Now try to change it

Four things you will want, in the order you will want them.

1. Make it faster
2. Change what it optimises
3. See what the gradients are doing
4. Stop part-way through and look

Try each one now, in the notebook, before the next slide.

Wall 1 · it is slow, and there is nowhere to move it

125 s

20 epochs, 55,000 images, 266,610 parameters, one CPU.

> **THERE IS NO `device=`**
>
> Scikit-Learn does not support GPUs, and says so in its own FAQ: it is a deliberate scope decision, not an oversight. The whole of Part II is going to need one.

Wall 2 · you cannot change the loss

```
>>> MLPClassifier(hidden_layer_sizes=(300, 100), loss="mae")
TypeError: MLPClassifier.__init__() got an unexpected
keyword argument 'loss'
```

Cross-entropy is hard-coded. That is fine until the operator says “calling a coat a shirt is cheap, calling a bag a sandal is expensive” — a weighted loss, which is one line in a framework that lets you write one.

***✗* And that is the ordinary case, not an exotic one.**

Wall 3 · you cannot see a gradient

```
>>> [a for a in dir(clf) if "grad" in a.lower()]
[]
```

Backpropagation ran 20 times per image and computed 266,610 partial derivatives on every batch. None of them was kept.

> **WHY THAT MATTERS IN THREE LECTURES**
>
> Lecture 13 builds a twenty-layer network whose loss barely moves. The diagnosis is a per-layer gradient statistic. You cannot compute it here.

Wall 4 · the smallest unit of control is one epoch

```
>>> inspect.signature(MLPClassifier.partial_fit)
(self, X, y, classes=None)
```

`partial_fit` is the finest handle there is, and one call is one complete pass over everything you hand it.

- No per-batch loss
- No hook between the forward pass and the backward pass
- No way to clip a gradient, or to skip a bad batch, or to log anything

The problem underneath all four

> **THE LOOP WAS WRITTEN FOR YOU**
>
> `fit()` contains a `for` loop over epochs and batches that you did not write, cannot see, and cannot enter. Every one of the four walls is the same wall.

This is not a criticism of Scikit-Learn. `MLPClassifier` is documented as a convenience, and for a tabular problem with a standard loss it is the right tool. It is the wrong tool from here on, and now you know precisely why.

The notebook for this lecture

Open Lecture 9 in Colab

- **Runs on free CPU.** Fashion-MNIST is subsampled so the whole notebook finishes in a few minutes
- The parameter-count arithmetic, checked against what the fitted model reports — the one place in this lecture where you can verify a number on paper first
- The architecture tables applied: same corpus, three output layers, three losses

> **WHAT TO CHANGE**
>
> Remove the activation — set it to `identity` — and refit. Accuracy collapses towards what a linear model gets, which is the derivation above, measured.

What we did

1. Derived a layer: one matrix product, one elementwise non-linearity — and showed that without the non-linearity any stack collapses to a single affine map
2. Met the perceptron, saw what it cannot separate, and the multilayer perceptron that can
3. Read the architecture tables: output units, output activation and loss, for regression and for classification
4. Fitted a network with `MLPClassifier` and tuned it by hand
5. Named four things that object will not let you do — and saw that all four are the same thing: the training loop is written for you, and you cannot enter it

In the book

Topic	Chapter
Biological neurons, the TLU, the perceptron	9
The perceptron learning rule, XOR, the MLP	9
Activation functions and why the step had to go	9
Regression and classification MLPs, the architecture tables	10
Softmax and cross-entropy	4, 10
Fashion MNIST	9
Confusion matrix, per-class recall	3

NOT PRESENTED — FOR AFTERWARDS

Exercises

Lecture 9

five, in the style of the written paper

Exercises — Lecture 9

1 A multilayer perceptron with the activation removed is written as $W_3 W_2 W_1 x$. State what function class it can represent, and why depth then buys nothing. [4]

2 Give the parameter count of a dense layer from m inputs to n units, and evaluate it for the first layer of a network on 28×28 images with 300 units. [4]

3 A single TLU cannot represent XOR. State the property of XOR that prevents it, and the smallest change to the network that fixes it. [4]

Solutions on Lecture 10’s deck.

Exercises — Lecture 9 (continued)

4 Fashion-MNIST is exactly balanced across ten classes. State the trivial baseline's accuracy, and why balance is what makes accuracy defensible here. [3]

5 An architecture sweep finds the best hidden-layer size on this dataset. State what that result does and does not transfer to. [4]

Solutions on Lecture 10's deck.

THE FIVE SET AT THE END OF LECTURE 8

Solutions

Lecture 8

not part of this lecture

Solutions — Lecture 8

1. PCA is derived by maximising $\|XW\|_F^2$ subject to $W^\top W = I$. State why the data must be centred…

✓ Without centring the leading direction points at the mean, not at the direction of greatest variance.

- The objective measures distance from the origin, and only after centring is that the same as spread
- On the Olivetti faces the uncentred first component is the mean face

2. The Johnson–Lindenstrauss bound, evaluated for 400 images at $\varepsilon = 0.1$, asks for more…

✓ The bound is vacuous here; the method still works.

- JL is a worst-case guarantee over all point sets, and this one is not worst case
- A bound that does not bind is not evidence against the technique — measure the distortion instead

Solutions — Lecture 8 (2)

3. A pipeline fits PCA on the whole dataset and then splits. Name what leaked, and say whether the leak grows or…

✓ The components leaked; the damage shrinks as the corpus grows.

- The principal directions are fitted parameters, so fitting them on test rows lets those rows shape their own representation
- With more rows each individual test row moves the components less

4. Inertia always falls as k rises, so it cannot choose k. State what silhouette adds, and one situation…

✓ Silhouette compares within-cluster to nearest-other-cluster distance, so it can fall. It fails when the clusters are not compact and separated.

- Inertia is monotone by construction: more centres can only shorten distances
- Silhouette assumes the geometry k-means assumes, so elongated or nested clusters defeat both

Solutions — Lecture 8 (3)

5. You have 4,096-dimensional face vectors and are told two faces are “close”. Explain why that…

✓ Distance is not identity — lighting and pose move a face further than identity does.

- Pixel distance is dominated by whatever varies most, and that is illumination
- The metric has to be one under which the thing you care about is the thing that varies

LECTURE 10 · GÉRON CH. 10

PyTorch

Backpropagation as reverse-mode automatic differentiation, derived
Tensors, autograd, and the training loop written out in full
Two bugs that never raise an exception

Run the Lecture 9 notebook first